

Руководство по сборке файла и разрешению коллизий

Специальная монография по самой насыщенной части декодера DataShield — сборке файла из накопленных секторов и разрешению коллизий версий. Документ дополняет `AlgorithmGuide.ru.md` (раздел 9 даёт краткий обзор): здесь та же тема разобрана формально — постановка задачи, структуры данных и их инварианты, пошаговые алгоритмы, математика полного перебора, эвристической прокрутки и подбора подмножества томов, проработанные примеры и точные границы гарантий. Контекст (формат пакета, хеши H5/D3, накопление) предполагается известным. Инженерный пересказ того же материала с совпадающей нумерацией разделов — `AssemblyGuide.Engineer.ru.md`; вводное изложение «с нуля», с разжёвыванием каждого шага и числовыми трассами, — `AssemblyGuide.Tutorial.ru.md`.

1. Постановка задачи

Пусть слот приёма `ReceptionSlot` создан заголовком с полями:

- `N` — число data-томов, `M` — число ECC-томов, `T = N + M`;
- `FileSize` — размер файла в байтах;
- `H3` — SHA-256 содержимого исходного файла (32 байта);
- `H5` — хеш заголовка, сид хеша секторов.

К моменту сборки накопитель находится в состоянии

```
для каждого номера  $i \in [0, T)$ :  $V_i = \{(\text{payload}_1, c_1), \dots, (\text{payload}_k, c_k)\}$ 
```

— множество **версий** payload сектора `i` со счётчиками подтверждений `c_j ≥ 1`; для отсутствующих секторов `V_i = ∅`.

Задача сборки. Найти выборку `s: [0, T) → payload ∪ {⊥}` (`⊥` — том не принят), для которой

1. все первые `N` томов определены напрямую **или** восстановимы по ECC из выборки;
2. `SHA-256(trim(s[0] || s[1] || ... || s[N-1], FileSize)) = H3`.

Арбитр подлинности — хеш всего файла — **не зависит от процесса приёма**: ни счётчики, ни порядок прихода, ни решения эвристик не влияют на итоговую проверку. Поэтому результат сборки бинарный: либо файл, бит-в-бит равный исходнику (доказательство — совпадение SHA-256), либо отказ (`null`). «Частично правильных» результатов не существует по построению.

2. Происхождение коллизий и модель угроз

Коллизия версий — состояние `|V_i| > 1`: под одним номером сектора накоплены два и более различных payload, каждый из которых прошёл проверку `D3 = Trunc9(SHA-256(H5 || D1 || D2))`.

Источники коллизий:

Источник	Вероятность/условие	Последствие
Случайная коллизия хеша	$\sim 2^{-72}$ на пару «payload-кандидат, известный H5»	Крайне маловероятно, но конечно
Целенаправленная подделка	H5 передаётся в потоке открыто; злоумышленник генерирует валидный сектор с чужим payload	Реализуемо; это расчётная модель угроз
Дубликаты и повторы	Payload совпадает → счётчик той же версии растёт	Коллизией не является

Подделка не может пройти финальную проверку (для этого нужен прообраз SHA-256 по H3), поэтому её максимум — отказ в сборке (DoS) либо затраты времени на перебор. Система защиты многослоенная:

- Счётчики подтверждений** — легитимные повторы усиливают истинную версию, подделке для победы нужны повторы подделки;
- Поиск комбинаций** — при равной правдоподобности перебор/прокрутка проверяют варианты;
- SHA-256 всего файла** — абсолютный арбитр.

3. Структуры данных и инварианты

Тип	Роль
<code>SectorVariant</code>	payload (64 Б) + <code>ConfirmationCount</code> (int, насыщение int.MaxValue)
<code>List<SectorVariant></code>	версии одного номера; инвариант — убывание счётчиков
<code>SortedDictionary<int, List<SectorVariant>></code>	состояние слота; ключ — номер сектора
<code>ChoicePoint</code>	номер сектора + живая ссылка на список версий + <code>TiedVariantCount</code>
<code>SectorVersionInfo</code>	публичный снимок версии (копия payload) для UI/диагностики

Инварианты списка версий:

- I1 (сортировка).** Счётчики нестрого убывают по индексу: `c_1 ≥ c_2 ≥ ...`. Поддерживается алгоритмом приёма (раздел 4).
- I2 (устойчивость равных).** Версии с равными счётчиками сохраняют текущий взаимный порядок; никакая операция не переставляет их между собой. Порядок может меняться только «прокруткой» равновероятной головы (раздел 11) — и снова без нарушения I1.

Живая ссылка `ChoicePoint.Variants` — не копия, а сам список слота: эвристическая прокрутка мутирует порядок версий непосредственно в слоте, поэтому построение выборки после прокрутки видит новое состояние. Снимков на каждую попытку не делается — их стоимость в горячем цикле поиска неоправдана.

4. Алгоритм приёма версии (`AddSector`)

Вход: номер `i`, payload `p` (64 Б). Шаги:

1. **Валидация.** `i ∉ [0, T)` или `|p| ≠ 64` → отказ (ложное распознавание исключено хешем, но проверка дешёвая и защищает API).
2. **Первый сектор номера.** `v_i = ∅` → создать список, добавить `(p, 1)`. Готово.
3. **Поиск совпадения.** Линейный проход по `v_i` с побайтовым сравнением payload:
 - **найдена версия** `(p, c)` → `c ← min(c + 1, int.MaxValue)` (насыщение), затем **всплытие**: пока у соседа сверху счётчик *строго* меньше, меняться местами. Через равные счётчики не переставляться — этим сохраняется I2. Готово.
 - **не найдена** → добавить `(p, 1)` **в конец** списка. Корректность: все существующие версии имеют `c ≥ 1`, новая — ровно 1, поэтому I1 не нарушается.

Сложность — $O(|v_i|)$ на копию сектора; $|v_i| = 1$ в подавляющем большинстве случаев, поэтому хеш-таблица по payload не используется (64-байтное сравнение дешевле лишней структуры и памяти).

Побочный эффект всплытия: версия, получившая повторное подтверждение, обгоняет менее подтверждённые — «лидер» списка `versions[0]` всегда самый подтверждённый. Именно на этом держится «предпочтительная выборка» сборки.

5. Конвейер сборки: общий план

TryAssemble

```
1. selected ← BuildPreferredSelection()           // versions[0] каждого номера
2. points ← BuildChoicePoints()                   // номера с равновероятной головой
3. points = ∅ ?
   да → одна попытка: прямая сборка → RS          // разделы 6–7
   нет ↓
4. C ← Π T_i                                     // c насыщением long.MaxValue
5. попытка №0 (нулевая комбинация) с замером времени t₀
6. оценка полного перебора t^ = t₀ · C
7. C ≤ MaxExhaustiveCombinations (100 000) и t^ ≤ TimeBudget (30 с) ?
   да → полный перебор (раздел 10)
   нет → эвристическая прокрутка (раздел 11)
8. результат L и M > 0 → подбор подмножества томов // раздел 12
```

Оба пути перебора на каждой итерации выполняют **одну и ту же** попытку сборки (раздел 6 → раздел 7), различаясь только способом порождения следующей комбинации. Отмена (`Cancellation.Token`) и бюджет времени проверяются между попытками. Стадия 3 включается последней, когда выбор версий исчерпан (или точек ветвления не было вовсе): пространство поиска меняется с версий на карты стираний — детали в разделе 12.

6. Одна попытка: прямая сборка

```
TryAssembleDirect(selected):
```

1. если $\exists i < N$: `selected[i] = 1` → отказ (не все данные на руках)
2. `buffer ← concat(selected[0..N])` // ровно $N \cdot 64$ байт
3. `TrimAndVerify(buffer)` // раздел 13

Сложность — $O(N \cdot 64)$ на конкатенацию плюс один SHA-256 по `FileSize` байт. Прямой путь покрывает случай полного или сверхполного приёма (потери закрыты повторами, а не ECC).

7. Одна попытка: RS-восстановление

Прямая сборка не прошла и `M > 0`:

1. Построить карту валидности `map[0..T)` (`true` — том присутствует в выборке: `selected[i] ≠ 1`) и массив слотов с выбранными payload.
2. Если присутствующих томов меньше `N` — восстанавливать не из чего, отказ.
3. `RsCodecAdapter.Decode(sectors, map, N)`:
 - все data-тома целы → **passthrough** (возврат как есть);
 - стёртых data-томов больше, чем доступных ECC-томов → отказ (кода не хватает);
 - иначе — решение системы над $GF(2^{16})$ с картой стираний: 32 независимые позиции символов, в каждой — столбец из `T` UInt16-символов; `Process` восстанавливает стёртые data-символы; `recovered`-тома собираются из восстановленных столбцов.
4. Проверить результат: ровно `N` томов по 64 байта; конкатенация `N · 64` → `TrimAndVerify` (раздел 13).

Ключевое обстоятельство: **ECC-тома участвуют в выборке наравне с data**, поэтому коллизия может стоять и на ECC-номере — подделанный ECC-том портит восстановление, финальный хеш не сходится, и поиск комбинаций обязан варьировать и ECC-версии тоже. RS-восстановление встроено в каждую попытку: перебор меняет только неоднозначные слоты, а дыры добираются ECC заново. Той же процедурой пользуется и стадия 3 (раздел 12) — с той разницей, что в карту, кроме отсутствующих, попадают **принудительно исключённые** тома (`forcedErasure`): RS обязан восстановить и их.

8. Точки ветвления и пространство поиска

Точка ветвления (`ChoicePoint`) — номер `i`, у которого максимальный счётчик достигается более чем одной версией:

$$T_i = |\{ v \in V_i : c(v) = \max_{\{u \in V_i\}} c(u) \}| \geq 2$$

`BuildChoicePoints` подсчитывает длину связной головы списка с счётчиком, равным `versions[0].ConfirmationCount`.

Пространство поиска — декартово произведение равновероятных голов:

```
Search = { (v_1, ..., v_m) : v_j ∈ первые T_j версий слота j }  
|Search| = C = Π T_j
```

Документированное ограничение: версии со счётчиком **строго меньше** максимального не участвуют в поиске ни в одной стратегии (одометр и прокрутка работают только с головой `T_j`). Если истинный payload оказался в меньшинстве подтверждений (подделка продублирована чаще истины), текущий вызов сборки завершится отказом. Это осознанный компромисс:

- вероятность хеш-валидной подделки — $\sim 2^{-72}$ на пакет, «победа» подделки требует ещё и перевеса подтверждений;
- продолжение приёма меняет счётчики — повторный вызов `TryAssemble` увидит новую конфигурацию;
- сам сектор может быть вообще не нужен: его восстановит RS.

9. Подсчёт комбинаций и выбор стратегии

`CountCombinations(factors)` — произведение `T_j` в `long` с **насыщением**: при ожидаемом переполнении возвращается `long.MaxValue` (в отчётах выводится как «> long.Max»). Насыщение гарантирует корректное сравнение с лимитом без арифметических исключений.

Выбор стратегии (`ShouldUseExhaustiveSearch`) требует **одновременно**:

1. `C ≤ MaxExhaustiveCombinations` — жёсткий комбинаторный лимит (по умолчанию 100 000);
2. `t_0 · C ≤ TimeBudget` — эмпирическая оценка по факту: нулевая попытка выполняется с замером (`Stopwatch`), её длительность умножается на `C`. Одна попытка включает SHA-256 и, возможно, RS — стоимость зависит от размера файла, поэтому статической оценки недостаточно.

Нарушение любого условия переводит поиск в эвристическую прокрутку.

10. Полный перебор (exhaustive, одометр)

Индексы выбора хранятся массивом `indexes[0..m)`, модули — `moduli[j] = T_j`. `AdvanceIndexes` — классический **одометр**: младшая позиция справа; инкремент с переносом; переполнение старшей позиции — комбинации исчерпаны (возврат `false`, индексы обнуляются).

Псевдокод:

TryExhaustiveSearch:

```
попытка №0 уже выполнена вызывающим кодом
for completed = 1 .. C-1:
    if токен отмены сработал          → исключение
    if таймер превысил TimeBudget     → отказ
    AdvanceIndexes(indexes, moduli) // следующая комбинация
    ApplyIndexes(selected, indexes) // подставить головы точек ветвления
    результат = Одна попытка         // разделы 6-7
    if результат ≠ 1 → вернуть
отказ
```

`ApplyIndexes` пишет `point.Variants[indexes[j]].Payload` в `selected[point.SectorNumber]`

— мутируется только локальная выборка, порядок версий в слоте не трогается.

Трассировка одометра для `moduli = (2, 3)` (индексы слева направо — старшая, младшая):

`(0,0) → (0,1) → (0,2) → (1,0) → (1,1) → (1,2) → сброс`

Полный перебор **полн** в пределах пространства поиска: при `C ≤ 100 000` и укладке в бюджет правильная комбинация (если она в головах `T_j`) будет найдена гарантированно.

11. Эвристическая прокрутка (rotation)

Когда перебор неразумно дорог, используется наблюдение о структуре типичной подделки: злоумышленник дублирует подделку во всех атакованных слотах **одинаково**, значит смещение истинных версий по рангу тоже систематично. Прокрутка проверяет «диагонали» пространства.

Алгоритм:

TryRotationSearch:

```
stateCount ← НОК(T_1, ..., T_m), ограниченный MaxHeuristicAttempts (100 000)
for state = 1 .. stateCount-1:
    RotateTiedVariants(points) // в каждом слоте первый равновероятный
                                // вариант переносится в конец головы T_j
    if бюджет/отмена          → отказ
    selected ← BuildPreferredSelection() // новое состояние слотов
    результат = Одна попытка
    if результат ≠ 1 → вернуть
отказ
```

Формальный анализ. После `k` прокруток в слоте `j` выбранная версия имеет исходный индекс `k mod T_j` (голова циклически сдвинута на `k`). Значит попытка с номером `k` проверяет комбинацию

$(k \bmod T_1, k \bmod T_2, \dots, k \bmod T_m)$

— «диагональ» пространства. Отсюда:

- **Период** последовательности попыток — $\text{НОК}(T_1, \dots, T_m)$; именно столько уникальных состояний (`CountRotationStates`, с ограничением сверху `MaxHeuristicAttempts`).
- **Достижимость.** Комбинация $(a_1, \dots, a_m)$ достижима $\iff$ существует k с $k \equiv a_j \pmod{T_j}$ для всех j — по обобщённой китайской теореме об остатках $\iff a_i \equiv a_j \pmod{\text{НОД}(T_i, T_j)}$ для всех пар. При попарно взаимно простых T_j $\text{НОК} = C = \prod T_j$ и прокрутка **эквивалентна полному перебору** (иного порядка обхода). При наличии общих делителей часть комбинаций недостижима — цена эвристики.
- **Систематическая подделка** (одинаковое смещение во всех атакованных слотах: a_j равны) лежит на диагонали $k = a$ — покрывается одной из первых попыток.

Пример $T = (3, 2)$, $\text{НОК} = 6$ — все 6 комбинаций достижимы:

```
k:      0      1      2      3      4      5
пары: A+X → B+Y → C+X → A+Y → B+X → C+Y    (затем повтор)
```

Пример $T = (2, 2, 2)$, $C = 8$, $\text{НОК} = 2$ — достижимы только $(0, 0, 0)$ и $(1, 1, 1)$: прокрутка проверяет 2 состояния из 8, но обе «систематические» диагонали — первой и второй попыткой.

Мутация состояния слота прокруткой безопасна для инвариантов: перенос внутри равновероятной головы не меняет мультимножество счётчиков и их порядок (I1), взаимный порядок версий с равными счётчиками меняется только внутри головы (I2 в расширенной формулировке). Последствия для повторных вызовов: порядок голов «прокручен», но сборка от этого не зависит — любая перестановка равновероятной головы одинаково допустима как стартовая.

12. Подбор подмножества томов (стадия 3)

Стадии 1–2 бессильны против **тихого повреждения**: сектор испорчен, но номер цел и усечённый хеш D3 сошёлся — случайная порча прошла 72-битную проверку (вероятность $\sim 2^{-72}$) либо payload согласован с хешем заранее (подделка); версия единственная. Точки ветвления нет; RS по полной карте принимает испорченный том за истину; каждая попытка честно возвращает `null`. Стадия 3 меняет пространство поиска: варьируется не выбор версий, а **карта стираний**. Формально фиксируются множества D (пропущенные data-тома), A (принятые ECC-тома) и **маска исключения** $X \subseteq [0, T)$:

```
map[i] = (том i принят  $\wedge$  i  $\notin$  X)
осуществимость:  $0 < |D| + |X \cap \text{data}| \leq \min(|A| - |X \cap \text{ECC}|, \text{MaxErasedDataVolumes})$ 
попытка(X) = RS-Decode(map) → TrimAndVerify // раздел 13
```


Первое условие отбрасывает маски без стёртых data (RS-passthrough повторил бы прямую сборку), второе — превышающие ECC-запас или кап E. Планировщик `SubsetMaskPlanner.Plan` строит ленивый поток осуществимых масок по уровням подозрительности:

1. **База** — все коллизионные слоты сразу. Их версии больше не перебираются: тома исключаются и восстанавливаются RS целиком (конфигурация, которой не было ни в одной попытке стадий 1–2). База неосуществима → план пуст: остаточные коллизии стадия не покрывает.
2. **Уровень 1** — база плюс одиночное исключение каждого присутствующего неколлизионного тома. Порядок — по подозрительности: по возрастанию подтверждений головы, внутри равных — псевдослучайно (тасование Фишера–Йетса на Mt19937 с сидом от H5 — детерминизм между вызовами и независимость от порядка прихода пакетов).
3. **Уровни 2..MaxExtraExclusionLevel** — сочетания из `ShortlistSize` первых (наиболее подозрительных) кандидатов: пары, тройки, ...; биномиальные коэффициенты уровней насыщаются на `long.MaxValue`.

Стоимость попытки — $\text{Init} \sim O(E^2 \cdot N) + 32 \times \text{Process} \sim O(E \cdot N) + \text{SHA-256}(N \cdot 64)$, где E — суммарное число стёртых data-томов (пропущенные + исключённые); исключение ECC-тома на E не влияет, но уменьшает ECC-запас. Калибровка бенчмарком (`RsRaid16Demo.bench`): при $E \leq 32$ даже максимальное поле ($T = 65\,535$) даёт десятки попыток в секунду; при $E \geq 128$ попытка стоит секунды. Отсюда отдельные лимиты: кап E (`MaxErasedDataVolumes`) ограничивает цену одной попытки, а `MaxAttempts` / `TimeBudget` — их число. Отмена и бюджет проверяются между попытками; прогресс — «подбор томов, попытка: k / оценка».

Гарантия уровня 1: если испорчен ровно один том вне коллизионной базы и лимиты не исчерпаны, маска с его исключением будет сгенерирована и попытка восстановит истину. Для уровней ≥ 2 аналогичное утверждение условно: все повреждённые тома должны попасть в шорт-лист, а их число — не превысить `MaxExtraExclusionLevel`.

13. Верификация результата и гигиена памяти (`TrimAndVerify`)

```
TrimAndVerify(buffer):  
    1. result ← buffer[0..FileSize)           // последний том добивался нулями —  
                                              // хвост отбрасывается  
    2. если SHA-256(result) = H3 → вернуть result  
    3. иначе: очистить buffer и result, вернуть ⊥
```

Теорема корректности. Если сборка вернула непустой результат, его SHA-256 совпадает с H3 заголовка, принятого до начала сборки. При криптографической стойкости SHA-256 это означает совпадение с исходным файлом бит-в-бит. Обратное неверно: отказ не доказывает отсутствия данных (мог не хватить бюджета), но никогда не выдаёт неверный файл.

Очистка буферов при неудаче — гигиена памяти: содержимое неудачной попытки (возможно, содержащее подделку) не задерживается в куче дольше необходимого.

14. Метрики слота и наблюдаемость

Член	Смысл
<code>HeaderReceptionCount</code>	копий заголовка (насыщение <code>int.MaxValue</code>)
<code>ReceivedSectorCount</code>	номеров $s \geq 1$ версией
<code>ReceivedSectorCopyCount</code>	всех копий секторов (сумма счётчиков версий)
<code>CollisionSectorCount</code>	номеров $s > 1$ версией
<code>Coverage</code>	доля принятых номеров от T, %
<code>BuildValidityMap</code>	<code>bool[T]</code> : номер принят
<code>FormatValidityMap</code>	'█' принят / '██' коллизия / '░░' пропуск
<code>BuildCollisionMap</code>	номер → кратность коллизии (только > 1 версии)
<code>GetSectorVersions(n)</code>	снимок версий с копиями payload в порядке предпочтительности

Прогресс сборки троттлится по целому проценту и не превышает 99 до фактического завершения; успешный исход репортится как 100 (`AssemblyFinished`). Попытки внутри поиска прогресс не публикуют — счётчик идёт по комбинациям/состояниям/маскам («полный перебор: k / C», «подбор томов: k / оценка»).

15. Параметры и пределы (`SectorVersionSearchOptions`)

Параметр	Умолчание	Смысл
<code>MaxExhaustiveCombinations</code>	100 000	жёсткий лимит C для полного перебора
<code>MaxHeuristicAttempts</code>	100 000	лимит состояний прокрутки (включая нулевое)
<code>TimeBudget</code>	30 с	мягкий бюджет; проверяется между попытками

Стадия 3 (раздел 12) настраивается группой `SubsetSearch` (`VolumeSubsetSearchOptions`):

Параметр группы <code>SubsetSearch</code>	Умолчание	Смысл
<code>arch</code>		
<code>MaxAttempts</code>	100 000	потолок попыток стадии (с бюджетом срабатывает раньше первый)
<code>TimeBudget</code>	30 с	мягкий бюджет стадии
<code>MaxErasedDataVolumes</code>	32	кап E — стёртых data-томов на одну попытку
<code>ShortlistSize</code>	64	шорт-лист подозрительных томов для уровней $t \geq 2$
<code>MaxExtraExclusionLevel</code>	3	максимальный размер дополнительных исключений

Ограничение на переполнение: `CountCombinations` насыщается на `long.MaxValue` ; `CountRotationStates` ограничивает НОК сверху лимитом попыток (переполнение произведения)

контролируется через $\text{НОК}(a,b) = a / \text{НОД}(a,b) \cdot b$ с проверкой делителя); биномиальные коэффициенты уровней исключений насыщаются на `long.MaxValue`.

16. Проработанные примеры

16.1. Бесколлиззионный приём

Файл 100 байт, ECC 0%: $N = 2, M = 0, T = 2$. Приняты оба сектора (по одному разу). `points = 0` → одна попытка: `selected = [v0, v1]`, прямая сборка `128 байт → trim(100) → SHA-256 = H3` → успех.

16.2. Одна точка ветвления, $T_1 = 2$

Сектор 0 имеет версии $A (c=3)$ и $B (c=3)$. $C = 2$, таймер t_0 . Если $2 \leq 100\,000$ и $2 \cdot t_0 \leq 30$ → перебор: попытка №0 — A , попытка №1 (одометр $(0) \rightarrow (1)$) — B . Истина найдётся максимум на второй попытке.

16.3. Две точки, $T = (3, 2)$

$C = 6$, НОК = 6. Полный перебор и прокрутка дают **одинаковый** набор попыток (порядок разный): одометр идёт $(0,0) (0,1) (1,0) (1,1) (2,0) (2,1)$, прокрутка — диагонали $A+X \rightarrow B+Y \rightarrow C+X \rightarrow A+Y \rightarrow B+X \rightarrow C+Y$. Оба полны.

16.4. Три точки, $T = (2, 2, 2)$

$C = 8 >$ лимита не превышает, но пусть оценка времени провалила перебор → прокрутка: НОК = 2, проверяются только (A, X, P) и (B, Y, Q) . Шесть «смешанных» комбинаций не проверяются — если истина среди них, сборка честно откажет (в бюджете эвристики). Это цена масштабируемости: полный проход был бы в 4 раза дороже.

16.5. Подделка-победитель

Сектор 5: истина $A (c=2)$, подделка $B (c=5)$. Голова списка — одна версия B (максимальный счётчик единоличен) → точки ветвления нет, выборка всегда берёт B , финальный SHA-256 не сходится, отказ. Никакой перебор не поможет — A вне пространства поиска (раздел 8). Лечение на уровне системы: продолжать приём; если повторы поднимут A до равенства, следующий вызов сборки увидит точку ветвления.

16.6. Тихо повреждённый том (стадия 3)

$N = 64$, $M = 8$ (ECC-запас — 8 томов). Том 7 испорчен, но номер цел и D3 сошёл; версия единственная. Стадии 1–2: точек ветвления нет, единственная попытка скармливает RS полную карту, где испорченный том — истина; H3 не сходится. Стадия 3: база пуста (коллизий нет), уровень 1 исключает присутствующие тома по одному; на маске $\{7\}$ $E = 1 \leq 8$ — RS восстанавливает истину, хеш сходится. Одиночное повреждение покрывается уровнем 1

гарантированно (в пределах лимитов): маски уровня не зависят от того, какой именно том испорчен.

17. Сводка гарантий

Доказуемые свойства:

1. **Корректность результата.** Непустой исход $\implies \text{SHA-256} = \text{H}_3 \implies$ (стойкость SHA-256) бит-в-бит исходник.
2. **Полнота прямого пути.** Все data-тома приняты однозначно $\implies$ успех первой же попытки.
3. **Полнота перебора.** `C ≤ 100 000` и оценка времени в бюджете $\implies$ правильная комбинация в головах `T_j` будет найдена (кроме отмены/бюджета — тогда честный отказ).
4. **Эквивалентность прокрутки при взаимно простых** `T_j`. НОК = C, все комбинации достигаются.
5. **Отсутствие ложных результатов.** Любой путь завершается либо доказанным файлом, либо `null`.

Эвристические свойства (без гарантий):

6. Покрытие прокрутки при общих делителях — только диагонали НОК.
7. Версии-меньшинства вне пространства поиска текущего вызова.
8. Укладка в `TimeBudget` — эмпирическая, по факту нулевой попытки.
9. Одинокое тихое повреждение вне коллизийной базы покрывается уровнем 1 стадии 3 — при запасе ЕСС, капе Е и лимитах (иначе честный отказ).
10. Множественные тихие повреждения — только если все повреждённые тома попали в шорт-лист, а их число не выше `MaxExtraExclusionLevel`.

Числовая сводка: хеш сектора 72 бита (подделка $\sim 2^{-72}$), хеш заголовка 192 бита, арбитр — SHA-256; перебор $\leq 100\,000$ комбинаций, прокрутка $\leq 100\,000$ состояний, подбор $\leq 100\,000$ масок при капе Е = 32; бюджеты по 30 с (настраивается).